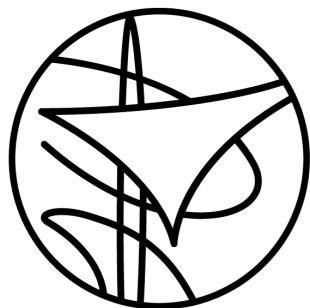




**ENSTA
BRETAGNE**

Rapport
projet info

Victor CACHAU,
Xavier CAPDEVILLE

FISE 2028

9 juin 2026

Table des matières

I	Introduction	3
II	Analyse du problème	3
II.1	Définition exacte du problème	3
II.2	Buts du projet	4
II.3	Domaines d'utilisation	4
III	Description des classes	4
III.1	GameConfig (config.py)	5
III.2	Kingdom (world/kingdom.py)	6
III.3	TurnPhase (core/game_state.py)	6
III.4	GameState (core/game_state.py)	6
III.5	GameEngine (core/game_engine.py)	7
III.6	BaseAI (core/ai/base_ai.py)	7
III.7	Unit et sous-classes (world/unit.py)	8
III.8	UnitSelector (world/selector.py)	8
III.9	City (world/city.py)	9
III.10	Construction et sous-classes (world/construction.py)	9
III.11	Map (world/map.py)	10
III.12	Tile (world/tile.py)	10
III.13	Systèmes de jeu	10
III.13.a	Movement (core/systems/movement.py)	10
III.13.b	Combat (core/systems/combat.py)	10
III.13.c	Economy (core/systems/economy.py)	11
III.13.d	Visibility (core/systems/visibility.py)	11
III.14	Interface utilisateur	11
III.14.a	Menu contextuel multi-catégories	11
IV	Description des méthodes importantes	12
IV.1	Génération de la carte : Map (world/map.py)	12
IV.1.a	Étape 1 : Échantillonnage de Poisson	12
IV.1.b	Étapes 2 et 3 : Attribution des biomes et partition de Voronoï	12
IV.1.c	Étapes 4 et 5 : Nettoyage morphologique	13
IV.1.d	Étapes 5 et 6 : Traitement des zones d'eau	13
IV.1.e	Étape 7 : Graphe de voisinage et validation	13
IV.2	Dijkstra : Movement.dijkstra_reachable	14
IV.3	Résolution du combat : Combat.compute_damage	14
IV.4	Expansion des villes : City.expend_territory	15
IV.5	Cycle de tour : GameEngine.end_turn	15
IV.6	Fondation d'une ville : GameEngine.found_city	15
IV.7	Placement initial : GameEngine.setup_start_units	15
IV.8	Bruit de Perlin : utils/noise.py	16
V	Description des tests et des résultats	16
V.1	Stratégie de test	16
V.1.a	Architecture de la suite de tests	16
V.1.b	Défis techniques et solutions	17
V.1.c	Lancement de la suite	17
V.1.d	Tests manuels complémentaires	17

V.2 Résultats obtenus	18
V.2.a Génération de carte	18
V.2.b Mécaniques de jeu	18
V.2.c Interface graphique	18
VI Conclusion	19
Annexes	22
A – Arbre des fichiers du projet	22
B – Fichier config.py complet	22
C – Biomes, coûts et modificateurs	23
D – Ressources, constructions et productions	24
E – Algorithme de bruit de Perlin	25
F – Captures d’écran annotées	26

I. Introduction

Ce rapport présente le projet informatique réalisé dans le cadre de l'UE 2.1 à l'ENSTA Bretagne. L'objectif est de concevoir et d'implémenter un jeu de stratégie de type 4X (*eXplore*, *eXpand*, *eXploit*, *eXterminate*) en Python, jouable au tour par tour avec une interface graphique.

Le projet, baptisé **Imperium Novum**, modélise un monde procédural sur lequel un joueur déplace des unités, fonde des villes, exploite des ressources et entre en conflit avec des adversaires. L'implémentation s'appuie sur la bibliothèque `pygame` pour le rendu graphique et `scipy` pour la génération procédurale de la carte.

L'architecture repose sur un système de **royaumes** (Kingdom) : chaque acteur du jeu (joueur humain ou intelligence artificielle) est encapsulé dans un royaume identifié, ce qui prépare un support multi-IA extensible. Un cadre abstrait (`BaseAI`) est déjà en place ; les contrôleurs d'IA concrets se branchent sur ce cadre via un cycle de tour Joueur → IA → Fin de tour.

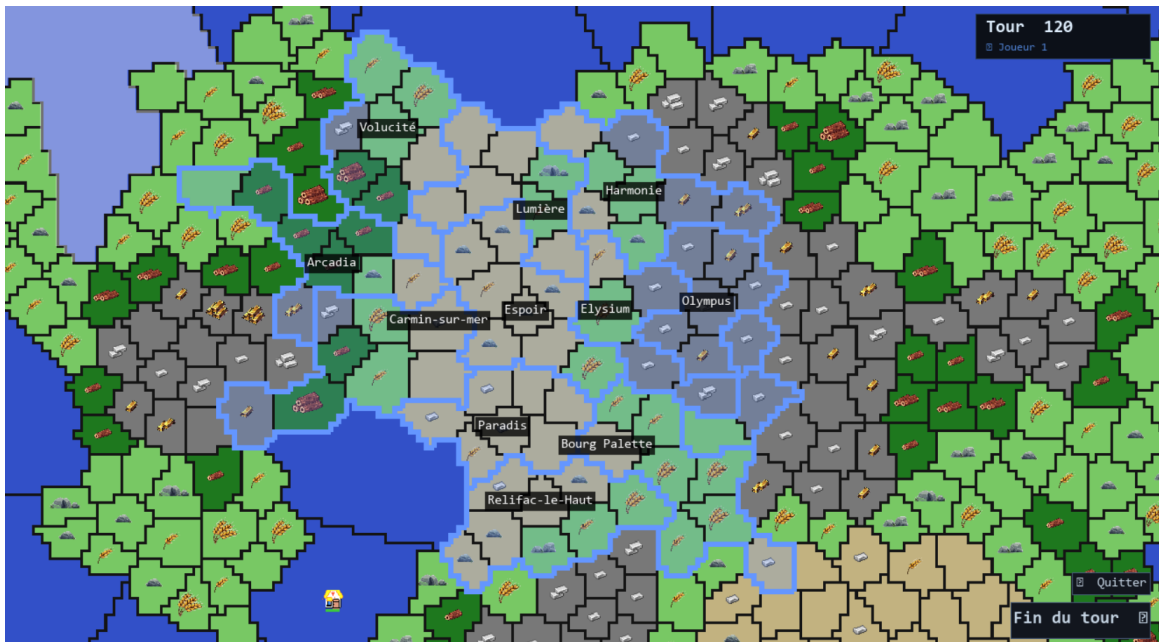


FIGURE 1 – Interface générale de *Imperium Novum* : carte procédurale avec villes, unités et indicateur de tour (tour 120)

II. Analyse du problème

II.1 Définition exacte du problème

Un jeu 4X est un genre de jeu de stratégie dans lequel chaque joueur développe une civilisation selon quatre axes fondamentaux :

1. **eXploration** : découvrir la carte progressivement à travers un brouillard de guerre ;
2. **eXpansion** : fonder des villes et étendre leur territoire ;
3. **eXploitation** : collecter des ressources et construire des bâtiments ;
4. **eXtermination** : attaquer les unités et villes adverses.

Le problème consiste à modéliser informatiquement ces quatre mécaniques de façon cohérente, dans un système tour par tour. La carte est une grille de provinces (*tuiles*) de types variés, chacune

pouvant contenir des unités et des ressources. Plusieurs systèmes interagissent simultanément : la gestion du mouvement, du combat, de l'économie et de la visibilité.

II.2 Buts du projet

Les objectifs définis pour ce projet sont les suivants :

- Concevoir une architecture orientée objet claire et modulaire ;
- Générer procéduralement une carte de jeu variée et jouable ;
- Implémenter les mécaniques fondamentales : déplacement, combat, production, exploration ;
- Réaliser une interface graphique interactive permettant de jouer effectivement une partie ;
- Respecter des règles de jeu équilibrées (coûts de terrain, triangles d'efficacité des unités, expansion des villes) ;
- Fournir une architecture permettant l'ajout d'IA ennemies.

II.3 Domaines d'utilisation

Ce projet relève de plusieurs domaines informatiques :

Algorithmique de graphes : pathfinding avec Dijkstra sur un graphe de provinces, BFS pour la visibilité et l'expansion des villes, transformée de distance pour la génération de super-tuiles d'eau.

Génération procédurale : bruit de Perlin multi-octaves pour les biomes, échantillonnage de Poisson pour les centres de provinces, diagrammes de Voronoï pour la partition de la carte.

Programmation orientée objet : hiérarchie de classes (unités, villes, constructions, royaumes), patron Facade pour le moteur de jeu, pattern Factory pour les unités, classe abstraite **BaseAI** pour les contrôleurs IA.

Infographie 2D : rendu Pygame avec surfaces mises en cache par couches, système de caméra avec zoom et panoramique, interface utilisateur (barres, menus contextuels multi-catégories).

III. Description des classes

La figure 2 résume les relations entre les classes principales du projet.

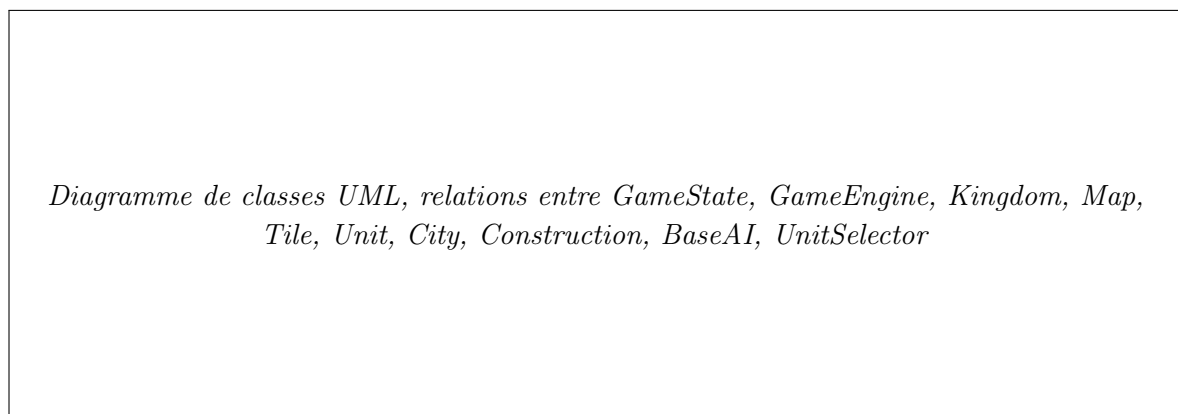


FIGURE 2 – Diagramme de classes UML de *Imperium Novum*

Le tableau suivant résume l'organisation modulaire du projet :

Module	Rôle
<code>run_project.py</code>	Point d'entrée (délègue à <code>ui/main.py</code>)
<code>config.py</code>	Constantes globales de configuration
<code>core/</code>	Moteur du jeu
<code>game_state.py</code>	État global du jeu + phases de tour
<code>game_engine.py</code>	Orchestrateur (façade)
<code>systems/</code>	Systèmes fondamentaux
<code>movement.py</code>	Système de déplacement
<code>combat.py</code>	Système de combat
<code>economy.py</code>	Système économique
<code>visibility.py</code>	Brouillard de guerre
<code>ai/base_ai.py</code>	Interface abstraite des contrôleurs IA
<code>world/</code>	Génération et stockage de la carte
<code>map.py</code>	Génération et stockage de la carte
<code>tile.py</code>	Province individuelle
<code>unit.py</code>	Hierarchie des unités + factory
<code>city.py</code>	Gestion des villes
<code>kingdom.py</code>	Représentation d'un royaume (joueur ou IA)
<code>construction.py</code>	Bâtiments (Ferme, Mine, Route, Scierie)
<code>resources.py</code>	Enumération des types de ressources
<code>biome.py</code>	Enumération des biomes
<code>selector.py</code>	Sélection d'unité, zones accessibles/attaquables
<code>ui/</code>	Logique du rendu graphique
<code>main.py</code>	Boucle principale du jeu
<code>renderer.py</code>	Pipeline de rendu Pygame
<code>camera.py</code>	Caméra avec zoom et panoramique
<code>ui_manager.py</code>	Gestionnaire de l'interface utilisateur
<code>utils/noise.py</code>	Bruit de Perlin

FIGURE 3 – Tableau récapitulatif des modules et de leurs rôles dans le projet

III.1 GameConfig (config.py)

GameConfig est une classe de configuration regroupant toutes les constantes du jeu sous forme d'attributs de classe. Elle évite la dispersion des paramètres dans le code et centralise les réglages de la carte, des boutons, de l'interface et des villes.

Listing 1 – Extrait de `config.py`

```

1 class GameConfig:
2     TILE_SIZE = 2
3     TILE_AVG_AREA = 35          # Taille moyenne d'une province (en
        cellules)
4     HEIGHT = 250                # Hauteur de la carte en tuiles
5     WIDTH = (HEIGHT * 16) // 9  # Largeur 16:9
6     CITY_MIN_TURN_EXTENTION_AVAILABLE = [3, 5, 8, 13, 21, 34, 53, 89]
7     CITY_EXTENSION_RADIUS = 3

```

```

8  SIDEBAR_WIDTH = 210
9  STATUS_H = 52

```

III.2 Kingdom (world/kingdom.py)

Kingdom est une dataclass représentant un royaume, qu'il soit contrôlé par un joueur humain ou par une IA. Chaque royaume possède une identité visuelle (couleur RGB) et, s'il est IA, un dictionnaire `ai_params` calibrant son arbre de décision.

Attributs :

- `kingdom_id` : identifiant unique (0 = joueur humain) ;
- `name` : nom du royaume (ex. « Joueur 1 », « Barbares ») ;
- `color` : tuple RGB utilisé pour la couleur d'affichage UI ;
- `is_ai` : True si le royaume est contrôlé par une IA ;
- `ai_params` : paramètres de décision (`aggression`, `exploration`, `expansion`, `defense`).

III.3 TurnPhase (core/game_state.py)

Enumération à deux valeurs gérant la phase active dans le cycle de tour :

- `PLAYER_TURN` : le joueur humain peut interagir ;
- `AI_TURN` : une IA est en cours de jeu, les entrées joueur sont verrouillées.

III.4 GameState (core/game_state.py)

Classe centrale qui encapsule l'intégralité de l'état d'une partie en cours. Elle ne contient pas de logique métier, mais sert de structure de données partagée entre tous les systèmes.

Attributs principaux :

- `map` : l'objet Map généré procéduralement ;
- `units` : liste de toutes les unités en jeu ;
- `cities` : liste de toutes les villes fondées ;
- `kingdoms` : liste des objets `Kingdom` enregistrés ;
- `turn_order` : liste ordonnée des `kingdom_id` dans l'ordre de jeu ;
- `current_kingdom_idx` : index courant dans `turn_order` (utilisé pendant la phase IA) ;
- `phase` : valeur de `TurnPhase` indiquant la phase active ;
- `player_resources` : dictionnaire `{kingdom_id: {ressource: quantité}}` ;
- `turn` : compteur de tours ;
- `discovered`, `visibility` : bitmasks pour le brouillard de guerre ;
- `use_ti`, `use_fow` : drapeaux pour activer la *terra incognita* et le brouillard de guerre.

Propriétés calculées :

- `active_kingdom_id` : ID du royaume dont c'est actuellement le tour ;
- `is_player_turn` : True si le joueur humain peut agir.

III.5 GameEngine (core/game_engine.py)

Moteur de jeu central qui implémente un patron *Facade* : il délègue chaque action à un système spécialisé tout en gérant les effets de bord (mise à jour de la visibilité, retrait des unités mortes, incrémentation des tours).

Systèmes délégués :

- `self.movement` : classe statique `Movement` ;
- `self.combat` : instance de `Combat` ;
- `self.economy` : instance de `Economy` ;
- `self.visibility` : instance de `Visibility`.

Principales méthodes publiques :

- `register_ai(ai)` : associe un contrôleur `BaseAI` à un royaume IA ;
- `spawn_unit(unit_type, tile_id, owner)` : instancie et place une unité ;
- `remove_unit(unit)` : retire une unité du jeu et met à jour la visibilité ;
- `move_unit(unit, target_tile_id)` : déplace une unité via `Movement` ;
- `attack_unit(attacker, target_tile_id)` : résout un combat via `Combat` ;
- `found_city(colon, city_name)` : fonde une ville à l'emplacement d'un colon ;
- `build_construction(tile, name, player)` : construit un bâtiment sur une tuile ;
- `setup_start_units()` : place le colon initial de chaque royaume dans un coin de la carte ;
- `end_turn()` : clôture le tour du joueur et déclenche les tours IA.

La propriété `input_locked` retourne `True` pendant `AI_TURN`, ce qui permet à la boucle principale de bloquer toutes les entrées joueur le temps que les IA jouent.

III.6 BaseAI (core/ai/base_ai.py)

Classe abstraite (ABC) définissant l'interface que doit respecter tout contrôleur IA.

Listing 2 – Interface abstraite BaseAI

```

1 class BaseAI(ABC):
2     def __init__(self, kingdom_id: int, params: dict): ...
3
4     @abstractmethod
5     def play_turn(self, state: GameState, engine: GameEngine) -> None:
6         """Appelle une fois par tour ; doit utiliser engine.move_unit()
7         et engine.attack_unit() -- ne modifie pas state directement."""
8         ...
9
10    # Helpers lecture seule :
11    def get_own_units(self, state): ...
12    def get_own_cities(self, state): ...
13    def get_enemy_units(self, state): ...
14    def get_visible_tiles(self, state) -> int: ...
15    def is_tile_visible(self, state, tile_id) -> bool: ...

```

Pour ajouter une IA, il suffit de sous-classer `BaseAI`, d'implémenter `play_turn()`, puis d'appeler `engine.register_ai(MonIA(kingdom_id=1, params=...))` avant la boucle de jeu.

III.7 Unit et sous-classes (world/unit.py)

Unit est une classe de base abstraite définissant l'interface commune à toutes les unités. Elle ne doit pas être instanciée directement. Chaque sous-classe redéfinit les attributs de classe (constantes) décrivant ses caractéristiques.

Capacités spéciales des unités :

DASH : peut se déplacer et attaquer en un seul tour ;

ESCAPE : peut se déplacer après avoir attaqué ;

SCOUT : portée de visibilité étendue (2 couronnes de tuiles au lieu de 1) ;

FLY : ignore les coûts de terrain (traverse eau, montagne, forêt sans pénalité) ;

PERSIST : peut attaquer tant qu'il y a des ennemis à portée, même après avoir attaqué (contrairement à **DASH** qui ne peut attaquer qu'une fois) ;

WATER_AFFINITY : peut traverser et se poser sur des tuiles d'eau ;

LAND_AFFINITY : peut se déplacer sur des tuiles terrestres.

Le tableau 1 suivant récapitule les caractéristiques de chaque unité.

Type	ATK	DEF	PV	Mvt	Portée	Coût	Capacités
Soldier	5	8	100	3	1	8	—
Cavalry	12	6	90	5	1	150	DASH, ESCAPE
Archer	8	5	70	2	2	10	SCOUT
Colon	2	3	50	1	0	15	WATER_AFFINITY
Plane	20	5	50	10	10	500	FLY, SCOUT, WATER_AFFINITY
Boat	5	3	50	3	1	100	WATER_AFFINITY

TABLE 1 – Caractéristiques des unités

Une factory **UNIT_CLASS_MAP** associe chaque valeur de l'énumération **UnitType** à la classe concrète correspondante.

III.8 UnitSelector (world/selector.py)

Gestionnaire de la sélection d'unités dans l'UI. Calcule et mémorise lors de la sélection :

- **reachable_tiles** : tuiles accessibles au mouvement ce tour (affichées en bleu) ;
- **attackable_tiles** : tuiles contenant des ennemis à portée d'attaque (affichées en rouge).

Ces ensembles sont recalculés à chaque sélection en appelant **Movement.get_reachable_tiles()** et **Movement.get_attackable_tiles()**.



FIGURE 4 – Visualisation des zones de mouvement (bleu) lors de la sélection d’un archer

III.9 City (world/city.py)

Représente une ville fondée par un colon. Une ville contrôle un ensemble de tuiles (`tile_ids`) et produit des ressources chaque tour en fonction des tuiles et des bâtiments présents. L’expansion du territoire est déclenchée automatiquement à des âges prédéfinis : [3, 5, 8, 13, 21, 34, 53, 89] tours (suite de Fibonacci tronquée).

Attributs principaux :

- `center_tile_id` : tuile fondatrice ;
- `tile_ids` : territoire contrôlé (ensemble d’IDs de tuiles) ;
- `population`, `age` : état démographique et temporel ;
- `production` : dictionnaire {`ressource`: `quantité/tour`}.

La méthode `get_visibility_mask()` permet à une ville de contribuer à la visibilité de son royaume : toutes les tuiles du territoire et leurs voisines immédiates sont visibles.

III.10 Construction et sous-classes (world/construction.py)

Quatre types de bâtiments peuvent être construits sur les tuiles contrôlées par une ville. Le bonus de production est calculé dynamiquement à la construction en fonction de la ressource présente sur la tuile.

Bâtiment	Coût	Bonus calculé dynamiquement
Ferme	5 bois	+1 à +3 nourriture (selon FOOD1/2/3 présente)
Mine	2 pierre + 3 bois	+1 à +3 fer/or/pierre selon ressource minière
Route	1 pierre + 1 bois	+1 mouvement sur la tuile
Scierie	5 bois	+1 à +3 bois (selon WOOD1/2/3 présente)

TABLE 2 – Constructions disponibles et effets sur la production

La règle de placement autorise une route et un bâtiment non-route par tuile, mais pas deux bâtiments non-route. Les routes peuvent coexister avec n'importe quel autre bâtiment.

III.11 Map (world/map.py)

Classe centrale de la carte. Elle stocke le dictionnaire `tiles:{id: Tile}` et encapsule le pipeline de génération procédurale en sept étapes principales (voir section IV). Elle utilise une structure de graphe par liste d'adjacence pour permettre le pathfinding et la diffusion. La génération est entièrement pilotée par une `seed` unique garantissant le déterminisme.

III.12 Tile (world/tile.py)

Représente une province individuelle. Chaque tuile possède :

- un identifiant unique `id` ;
- un biome (`Biome`) et une ressource (`Resource`) ;
- une liste de cellules visuelles (`cells`) qui la composent ;
- un ensemble de voisins (`neighbors`) pour le graphe ;
- une liste d'unités présentes (`units`) et de constructions (`constructions`).

III.13 Systèmes de jeu

III.13.a Movement (core/systems/movement.py)

Classe statique gérant les déplacements. Utilise l'algorithme de Dijkstra sur le graphe de voisinage des tuiles avec des coûts variables selon le biome et le type d'unité. Fournit également `get_attackable_tiles()` calculant les tuiles ennemies à portée d'attaque (y compris la portée ≥ 2 des archers et avions).

III.13.b Combat (core/systems/combat.py)

Gère la résolution des combats selon une formule intégrant l'attaque, la défense, un modificateur de terrain et un modificateur de type d'unité (triangle d'efficacité). Gère également la mécanique de *dash* de la cavalerie : déplacement et attaque en une seule action.

III.13.c Economy (`core/systems/economy.py`)

Calcule la production des villes (en recalculant `calculate_production()` à chaque tour pour tenir compte des nouvelles constructions) et déduit les coûts d'entretien des unités en or à chaque fin de tour pour tous les royaumes actifs.

III.13.d Visibility (`core/systems/visibility.py`)

Met à jour les bitmasks de visibilité et de découverte à partir des positions des unités et des villes du joueur courant. Les unités de type SCOUT étendent la visibilité à deux couronnes de tuiles au lieu d'une.

III.14 Interface utilisateur

- **Camera** (`ui/camera.py`) : gère le panoramique et le zoom ($1\times$ à $25\times$) avec conversion de coordonnées monde/écran.
- **Renderer** (`ui/renderer.py`) : pipeline de rendu en couches (carte, ressources, bâtiments, unités, brouillard, interface). Les surfaces Pygame sont mises en cache et reconstruites uniquement si marquées comme *dirty*. Affiche aussi les nombres de dégâts animés lors des attaques.
- **UIManager** (`ui/ui_manager.py`) : gère la barre latérale gauche (ressources, boutons), la barre de statut basse, l'indicateur de tour animé, et les menus contextuels multi-catégories (Construction / Unités / Territoire).

III.14.a Menu contextuel multi-catégories

Un clic droit sur une tuile ouvre un menu contextuel dont les catégories disponibles dépendent du contexte :

Construction (bleu) : visible sur toute tuile terrestre (appartenant au joueur ou neutre) ; propose Ferme, Mine, Route, Scierie selon le biome et les constructions déjà présentes.

Unités (rouge) : visible uniquement sur les tuiles appartenant à une ville du joueur ; permet d'acheter Soldat, Archer ou Colon en dépensant de l'or.

Territoire (or) : visible sur les tuiles libres adjacentes à une ville du joueur ; permet d'annexer la tuile pour 10 pièces d'or.

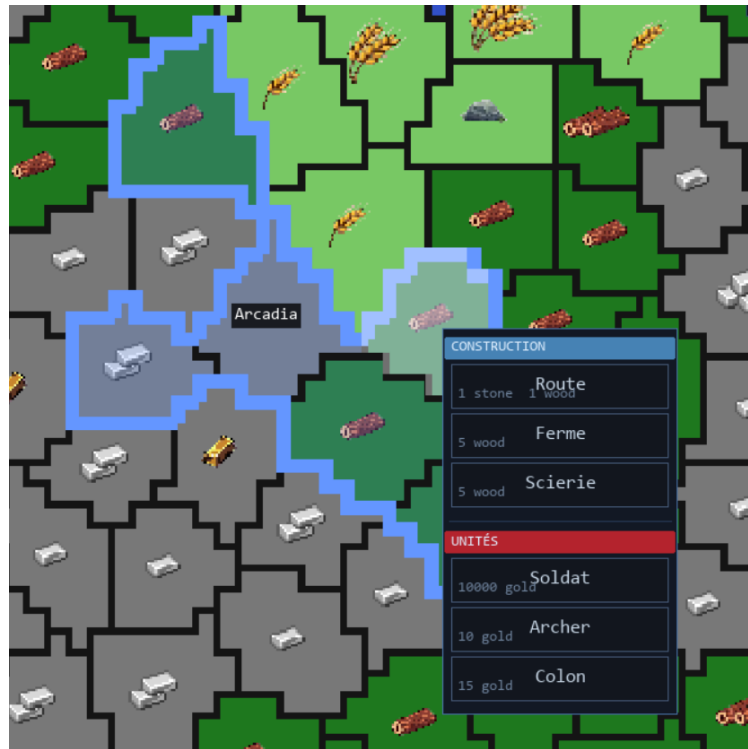


FIGURE 5 – Menu contextuel multi-catégories : Construction, Unités et Territoire

IV. Description des méthodes importantes

IV.1 Génération de la carte : Map (world/map.py)

La génération procédurale se déroule en sept étapes principales. On part d'une grille de cellules vierge qui sera ensuite partitionnée en provinces (tuiles) et classifiée en biomes. On appelle n le nombre de cellules de la grille (environ 110 000 pour une carte de 444×250 tuiles avec une taille).

Par soucis de simplicités, certaines étapes sont regroupées dans une même section du fait de la similarité de leur but, mais elles sont programmées de manière distincte.

IV.1.a Étape 1 : Échantillonnage de Poisson

Les centres de provinces sont distribués uniformément sur la carte avec une contrainte de distance minimale, en utilisant l'algorithme de Poisson Disk Sampling [2]. Une grille d'accélération spatiale assure une complexité en $O(n)$.

IV.1.b Étapes 2 et 3 : Attribution des biomes et partition de Voronoï

Un bruit de Perlin multi-octaves génère une carte d'élévation. Des seuils distincts classifient chaque cellule en biome (type de terrain) :

- Eau : $n < -0,225$
- Plaine : $-0,225 \leq n < 0,14$
- Forêt : $0,14 \leq n < 0,325$

- Montagne : $n \geq 0,325$
- Désert : Plaine avec indice de chaleur élevé (second bruit indépendant)

Un KD-tree (`scipy.spatial.KDTree`) assigne chaque cellule de la grille à la province la plus proche en $O(n \log n)$.

Puis chaque province obtient un biome majoritaire par vote de ses cellules, et une ressource est assignée aléatoirement selon le biome qui a été attribué à la province en question.

IV.1.c Étapes 4 et 5 : Nettoyage morphologique

Un BFS détecte les composantes connexes et fusionne les micro-provinces sous un seuil de taille minimum avec leurs voisines dominantes.

IV.1.d Étapes 5 et 6 : Traitement des zones d'eau

1. Suppression des clusters d'eau isolés trop petits.
2. Connexion des masses d'eau proches par des détroits.
3. Élargissement des corridors d'eau trop fins.
4. Fusion des tuiles d'eau en super-tuiles plus grandes (relaxation de Lloyd avec répulsion paramétrée par la distance au littoral).

IV.1.e Étape 7 : Graphe de voisinage et validation

Construction du graphe à 8-connexité entre tuiles adjacentes, puis validation de l'intégrité de la carte (cohérence bidirectionnelle entre cellules et tuiles).



FIGURE 6 – Exemple de carte générée procéduralement avec le pipeline Poisson–Voronoi–Perlin (plaine verte, forêt sombre, montagne grise, désert beige, eau bleue)

IV.2 Dijkstra : Movement.dijkstra_reachable

Listing 3 – Algorithme de Dijkstra pour le déplacement

```

1  @staticmethod
2  def dijkstra_reachable(map_, start_tile_id, max_movement, unit_type):
3      heap = [(0, start_tile_id)]
4      distances = {start_tile_id: 0}
5      visited = set()
6
7      while heap:
8          current_cost, current_tile_id = heapq.heappop(heap)
9          if current_tile_id in visited:
10             continue
11          visited.add(current_tile_id)
12          if current_cost > max_movement:
13             continue
14
15          for neighbor_tile_id in map_.tiles[current_tile_id].neighbors:
16              if neighbor_tile_id in visited:
17                 continue
18              terrain_cost = Movement.get_movement_cost(
19                  map_.tiles[neighbor_tile_id].biome, unit_type
20              )
21              if terrain_cost == float("inf"):
22                 continue
23              new_cost = current_cost + terrain_cost
24              if neighbor_tile_id not in distances or new_cost < distances[
25                  neighbor_tile_id]:
26                  distances[neighbor_tile_id] = new_cost
27                  heapq.heappush(heap, (new_cost, neighbor_tile_id))
28
29      return distances

```

Les coûts de terrain varient selon le biome (1,0 en plaine, 1,5 en forêt, 2,0 en montagne, $+\infty$ sur l'eau pour les unités terrestres) et sont modulés par des coefficients propres à chaque type d'unité. Les unités avec `FLY=True` (Avion) ignorent ces coûts et se déplacent librement sur toute la carte.

IV.3 Résolution du combat : Combat.compute_damage

La formule de dégâts intègre quatre facteurs :

$$\text{dégâts} = (\text{ATK} - \text{DEF} \times m_{\text{terrain}}) \times m_{\text{type}} \times \frac{\text{PV}_{\text{courant}}}{\text{PV}_{\text{max}}} \quad (1)$$

- m_{terrain} : modificateur de défense selon le biome du défenseur (montagne $\times 1,5$, forêt $\times 1,3$, plaine $\times 1,0$, désert $\times 0,9$, eau $\times 0,8$);
- m_{type} : modificateur d'efficacité selon le triangle Soldat $\succ$ Cavalerie $\succ$ Archer $\succ$ Soldat (valeurs 1,5 / 0,7);
- le ratio PV modélise l'affaiblissement d'une unité blessée.

La mécanique de **dash** de la cavalerie (attribut `DASH=True`) permet d'enchaîner un déplacement et une attaque en un seul tour. L'attribut `ESCAPE=True` lui permet en outre de se repositionner après avoir attaqué (sans avoir encore bougé).

IV.4 Expansion des villes : `City.expend_territory`

L'expansion utilise un BFS depuis la tuile centrale, limité au rayon `CITY_EXTENSION_RADIUS` (3 tuiles). Parmi les tuiles candidates libres (non-eau, non déjà attribuées), une tuile est tirée aléatoirement avec une probabilité inversement proportionnelle à sa distance :

$$P(\text{tuile}_i) = \frac{1/d_i}{\sum_j 1/d_j} \quad (2)$$

L'expansion n'est déclenchée que lors des tours où l'âge de la ville concernée appartient à la suite [3, 5, 8, 13, 21, 34, 53, 89].

IV.5 Cycle de tour : `GameEngine.end_turn`

Le cycle complet d'un tour comporte trois phases :

1. **Tour joueur** (`PLAYER_TURN`) : le joueur humain interagit jusqu'à appuyer sur « Fin du tour » (bouton ou touche Espace) ;
2. **Tours IA** (`AI_TURN`) : chaque royaume IA enregistré via `register_ai()` joue son tour en appelant `ai.play_turn(state, engine)`. Pendant cette phase, `input_locked` est `True` et tous les événements clavier/souris joueur sont ignorés ;
3. **Fin de tour** : mise à jour de l'économie (`Economy.update`), réinitialisation des drapeaux `has_moved/has_attacked`, vieillissement des villes et expansion éventuelle, recalcul du brouillard de guerre.

IV.6 Fondation d'une ville : `GameEngine.found_city`

Un colon positionné sur une tuile terrestre libre peut fonder une ville via un clic droit. La méthode vérifie les préconditions (type d'unité, existence de la tuile, absence de ville existante, biome non-eau), crée l'objet `City`, calcule sa production initiale, puis supprime le colon. Le nom de la ville est choisi aléatoirement dans une liste de noms prédéfinis, sans réutiliser les noms déjà pris par le même joueur.

IV.7 Placement initial : `GameEngine.setup_start_units`

À l'initialisation, chaque royaume reçoit un colon de départ positionné dans un coin de la carte selon l'ordre d'inscription : coin haut-gauche, bas-droite, haut-droite, bas-gauche. La méthode `get_corner_tile()` identifie la tuile non occupée la plus proche du coin demandé en triant les tuiles par somme/différence de coordonnées.

IV.8 Bruit de Perlin : `utils/noise.py`

Implémentation du bruit de Perlin standard avec table de permutation de Ken Perlin, courbe de lissage quintic $f(t) = 6t^5 - 15t^4 + 10t^3$, et génération de bruit fractionnaire Brownien (fBm) par superposition d'octaves :

$$\text{fBm}(x, y) = \sum_{k=0}^{n-1} p^k \cdot \text{Perlin}(l^k \cdot x, l^k \cdot y) \quad (3)$$

où p est la persistance et l la lacunarité. L'implémentation est une copie de la librairie `noise`[\[1\]](#) disponible avec `pip` mais non mise à jour depuis plusieurs années.

V. Description des tests et des résultats

V.1 Stratégie de test

Le projet dispose d'une suite de **334 tests unitaires automatisés** exécutés avec `pytest`, organisés dans le dossier `test/`. Ces tests couvrent l'ensemble des modules fonctionnels : modèle du monde, systèmes de jeu et utilitaires.

V.1.a Architecture de la suite de tests

La suite est structurée en miroir de l'architecture du projet :

`test/world/` — Couche modèle (6 fichiers, ≈ 200 tests) :

- `test_tile.py` : initialisation, centre de gravité, `is_water()`, ajout/suppression/itération d'unités, `clear_units()`, `__repr__` ;
- `test_unit.py` : statistiques de toutes les classes d'unités (Soldat, Cavalerie, Archer, Colonne, Bébé, Avion, Bateau), compteur d'ID global, `can_move()`, `reset_turn()`, masque de visibilité, fabrique `UNIT_CLASS_MAP` ;
- `test_construction.py` : bonus de production des bâtiments (Ferme, Mine, Scierie, Route) pour chaque niveau de ressource (1 à 3) ;
- `test_city.py` : auto-incrément d'ID, calcul de production par type de ressource, masque de visibilité, protection de la tuile centrale ;
- `test_kingdom.py` : dataclass `Kingdom`, valeurs par défaut, `__repr__` ;
- `test_resources.py` : énumération `Resource`, 16 membres, extraction du niveau (chiffres 1–3).

`test/core/` — Systèmes de jeu (4 fichiers, ≈ 120 tests) :

- `test_game_state.py` : royaumes, villes, phases de tour, accumulation du bitmask de visibilité (avec patch de `core.game_state.Map`) ;
- `test_movement.py` : coûts de déplacement par biome/unité, distances Dijkstra, tuiles accessibles, tuiles attaquables, `execute_move()`, coût vers une tuile cible ;
- `test_combat.py` : table d'efficacité (triangle Soldat/Cavalerie/Archer), calcul des dégâts (terrain, type, ratio PV), `can_attack()`, `resolve()`, `execute_attack()` ;
- `test_economy.py` : accumulation de la production de villes, déduction de l'entretien des unités pour le joueur actif uniquement ;
- `test_visibility.py` : application du masque de brouillard de guerre.

`test/utils/` — Utilitaires (1 fichier, ≈ 25 tests) :

- `test_noise.py` : interpolation linéaire, gradient 2D, plage du bruit de Perlin (borné en $[-1, 1]$), déterminisme, octaves multiples, table de permutation.

V.1.b Défis techniques et solutions

Isolation de Pygame. Le module `world/unit.py` appelle `pygame.init()` et `pygame.font.SysFont()` à l'import, ce qui provoque une erreur dans un environnement sans affichage. La solution adoptée est d'injecter un `MagicMock` dans `sys.modules["pygame"]` *avant* tout import de module projet, dans `test/conftest.py`. Ceci garantit que tous les tests s'exécutent sans interface graphique.

Substituts légers (MockMap, MockGameState). Les systèmes de jeu (Mouvement, Combat, Économie) opèrent sur un objet `GameState` contenant une vraie `Map` générée procéduralement, ce qui rendrait les tests lents et non déterministes. On leur substitue :

- `MockMap` : dictionnaire de tuiles créées à la main avec voisinage explicite ;
- `MockGameState` : état minimal avec ressources, joueur courant et bitmask de visibilité pré-initialisés.

Cette approche réduit le temps d'exécution de la suite complète à moins de 0,2s.

Compteurs globaux. `Unit._unit_counter` et `City._next_id` sont des attributs de classe (entiers auto-incrémentés). Une fixture `autouse=True` dans `conftest.py` les remet à zéro avant chaque test, garantissant l'isolation inter-tests.

Chemin relatif dans noise.py. `utils/noise.py` ouvre `"utils/perm.json"` avec un chemin relatif au répertoire courant. Le `conftest` effectue `os.chdir(PROJECT_ROOT)` au démarrage de `pytest` pour que ce chemin soit toujours résolu correctement.

V.1.c Lancement de la suite

Commande de lancement des tests

```
# Depuis la racine du projet
python3 -m pytest test/

# Avec rapport de couverture éédtaill
python3 -m pytest test/ -v --tb=short
```

V.1.d Tests manuels complémentaires

En parallèle des tests automatisés, une méthode de validation de bout en bout reste indispensable : lancer une partie, observer le comportement de chaque mécanique (déplacement, combat, fondation de villes, économie) et vérifier visuellement la cohérence. Les retours de débogage sont imprimés dans la console (`print`).

Un outil dédié, `perlin_visualizer.py`, permet de visualiser interactivement le bruit de Perlin et d'ajuster ses paramètres. La touche **R** régénère la carte en temps réel, ce qui a facilité l'itération sur les paramètres de génération.

V.2 Résultats obtenus

V.2.a Génération de carte

La génération procédurale produit des cartes cohérentes avec :

- une distribution réaliste des biomes (plaines, forêts, montagnes, déserts, océans) ;
- des provinces de taille homogène (cible : ≈ 35 cellules) ;
- des masses d'eau connectées entre elles via des détroits ;
- des ressources distribuées selon le biome (bois uniquement en forêt, fer/or en montagne, etc.).

V.2.b Mécaniques de jeu

- **Déplacement** : le pathfinding Dijkstra fonctionne correctement. La surbrillance des tuiles accessibles (bleue) est cohérente avec les points de mouvement et les coûts de terrain.
- **Combat** : la formule de dégâts est fonctionnelle. Le triangle d'efficacité (Soldat $\succ$ Cavalerie $\succ$ Archer $\succ$ Soldat) est respecté. Le dash de la cavalerie fonctionne.
- **Économie** : la production des villes est calculée chaque tour. L'entretien des unités est déduit. Une balance négative en or n'est pas encore pénalisée.
- **Visibilité** : le brouillard de guerre (FoW) et la *terra incognita* (TI) fonctionnent correctement. Le bitmask est suffisant pour la carte de démonstration. Les unités SCOUT (Archer, Avion) étendent la visibilité à 2 couronnes.
- **Fondation de villes** : un colon peut fonder une ville. L'expansion territoriale se déclenche aux bons tours et sélectionne aléatoirement une tuile voisine selon une loi de probabilité inverse à la distance.
- **Construction** : les bâtiments (Ferme, Mine, Route, Scierie) peuvent être construits depuis le menu contextuel si le joueur dispose des ressources nécessaires.
- **Achat de tuile** : une tuile neutre adjacente à une ville peut être annexée pour 10 pièces d'or via le menu contextuel (catégorie Territoire).
- **Achat d'unité** : un Soldat, Archer ou Colon peut être acheté directement depuis le menu contextuel sur les tuiles d'une ville du joueur.

V.2.c Interface graphique

L'IHM est fonctionnelle :

- panoramique fluide à la souris et au clavier (Z/S/Q/D) ;
- zoom de $1\times$ à $25\times$ centré sur la position du curseur ;
- barre latérale affichant les ressources et les boutons d'action avec animations lissées ;
- barre de statut affichant les informations de la tuile survolée (biome, ressource, constructions, unité) ;
- indicateur de tour animé (pulsation bleue en tour joueur, pulsation rouge en tour IA) ;

- rendu optimisé par mise en cache des surfaces Pygame (reconstruction uniquement si l'état a changé).



FIGURE 7 – Rendu du brouillard de guerre (*Fog of War*) et de la *Terra Incognita* : zones noires (inconnues), grises (déjà explorées), claires (actuellement visibles)

VI. Conclusion

Le projet **Imperium Novum** remplit ses objectifs principaux : il constitue un jeu 4X simplifié, jouable et structuré, avec une architecture orientée objet modulaire et une interface graphique interactive. Les quatre axes du genre (exploration, expansion, exploitation, extermination) sont représentés, même si certains restent partiellement implémentés.

Les points forts du projet sont la qualité du pipeline de génération procédurale de carte (Poisson + Voronoï + Perlin + post-traitements hydrographiques), le système de déplacement basé sur Dijkstra, et l'architecture en systèmes découplés coordonnés par le moteur. Le cadre multi-royaumes et l'interface abstraite **BaseAI** constituent une base solide pour l'ajout d'adversaires automatiques sans modifier le code existant.

Ce projet nous a surtout permis de réaliser qu'il existe un monde entre *coder* et *créer un jeu*. Fixer un coût d'entretien à une unité tient en une ligne de code ; équilibrer ce coût pour que la partie soit intéressante du début à la fin est un problème d'une toute autre nature. Il en va de même pour les conditions de victoire : écrire `if gold > 10000` est trivial, mais concevoir une fin de partie cohérente, qui ne lasse pas le joueur et qui reste ouverte jusqu'au bout, relève de la conception de jeu bien plus que de l'informatique.

L'implémentation d'une IA adverse illustre encore mieux ce fossé. Le défi ne venait pas du code — l'interface **BaseAI** est en place — mais du fait que nous sommes les concepteurs du jeu et

savons donc, par définition, comment y jouer parfaitement. Vouloir inculquer des stratégies à une IA sans passer par de l'apprentissage automatique revient à écrire à la main tout ce que nous faisons intuitivement, ce qui constituerait à lui seul un projet informatique entier.

Il faut également souligner que la génération procédurale de la carte est d'une complexité algorithmique telle qu'elle constitue elle aussi un projet à part entière. Le pipeline Poisson–Voronoi–Perlin, les post-traitements hydrographiques et les heuristiques de validation ont mobilisé plus de **30 heures de travail** à eux seuls.

Ce projet nous a confortés dans notre capacité à concevoir, maintenir et faire évoluer une base de code conséquente et multi-couches. Il nous a surtout fait prendre conscience que le développement va bien au-delà du terminal : le retour utilisateur est primordial, et c'est ce que les tests illustrent parfaitement. Une batterie de 334 tests ne peut pas reproduire les centaines d'actions imprévues qu'une vraie base de joueurs découvrirait en quelques heures de partie — deux bugs ont pourtant déjà été détectés à leur rédaction. Cela confirme que formaliser les tests est bénéfique dès maintenant, et qu'une couverture plus large reste un objectif pour les prochaines itérations.

Références

- [1] C. DUNCAN, *noise : Perlin noise library for Python*, <https://github.com/caseman/noise>, Version 1.2.2, last accessed 2026-06-07, 2015.
- [2] M. S. EBEIDA, A. A. DAVIDSON, A. PATNEY, P. M. KNUPP, S. A. MITCHELL et J. D. OWENS, « Efficient maximal poisson-disk sampling, » in *ACM SIGGRAPH 2011 papers*, Vancouver British Columbia Canada : ACM, juill. 2011, p. 1-12.

Annexes

A – Arbre des fichiers du projet

```

proj_info/
|-- run_project.py          # point d'entree
|-- config.py              # constantes globales (GameConfig)
|-- core/
|   |-- game_engine.py     # facade principale
|   |-- game_state.py      # etat mutable (GameState, TurnPhase)
|   |-- ai/
|       |-- base_ai.py     # interface abstraite BaseAI
|       |-- simple_ai.py   # implementation IA simple
|   |-- systems/
|       |-- combat.py
|       |-- economy.py
|       |-- movement.py
|       |-- visibility.py
|-- ui/
|   |-- main.py            # boucle principale pygame
|   |-- ui_manager.py      # rendu & menu contextuel
|   |-- sidebar.py
|-- utils/
|   |-- noise.py           # bruit de Perlin
|   |-- perm.json          # table de permutation de Ken Perlin
|-- world/
|   |-- biome.py           # enum Biome
|   |-- city.py            # classe City
|   |-- construction.py    # Farm, Mine, Scierie, Road
|   |-- kingdom.py         # dataclass Kingdom
|   |-- map.py             # generation de carte (Map)
|   |-- resources.py       # enum Resource
|   |-- selector.py        # UnitSelector
|   |-- tile.py            # classe Tile (province)
|   |-- unit.py            # Unit + sous-classes + UNIT_CLASS_MAP

```

B – Fichier config.py complet

Listing 4 – Fichier config.py – constantes globales

```

1 class GameConfig:
2     # --- Carte ---
3     TILE_SIZE = 2          # pixels par cellule de grille
4     TILE_AVG_AREA = 35     # surface moyenne d'une tuile Voronoi
5     HEIGHT = 250
6     WIDTH = (HEIGHT * 16) // 9
7
8     FPS = 60
9

```

```

10  SCREEN_WIDTH  = WIDTH  * TILE_SIZE
11  SCREEN_HEIGHT = HEIGHT * TILE_SIZE
12
13  # --- Generation de la carte ---
14  LOG_MAP_GENERATION      = True
15  BORDER_STRENGTH        = 2.75
16  OCEANIC_COEFF          = 1.5      # plus eleve = moins d'eau
17  OCEANIC_OFFSET         = 0.1
18  DESERT_HEAT_THRESHOLD  = 0.15
19  MIN_WATER_CLUSTER_SIZE = 2      # clusters <= 2 tuiles convertis en
    terre
20  WATER_CONNECT_MIN_SIZE = 8      # taille min d'une masse d'eau a relier
21  WATER_BRIDGE_MAX_HOPS  = 3      # distance max pour relier deux masses d
    'eau
22  WATER_TILE_SIZE_COEFF  = 8      # fusion de tuiles d'eau proches
23
24  # --- Couleurs des biomes (R, G, B) ---
25  BIOME_COLORS = {
26      Biome.BLANK:      ( 0,  0,  0),
27      Biome.WATER:      ( 50,  80, 200),
28      Biome.PLAIN:      (120, 200, 100),
29      Biome.FOREST:      ( 30, 120,  30),
30      Biome.MOUNTAIN:    (120, 120, 120),
31      Biome.DESERT:      (194, 178, 128),
32  }
33
34  # --- Interface ---
35  SIDEBAR_WIDTH          = 210      # largeur barre laterale gauche
36  STATUS_H               = 52      # hauteur barre de statut bas
37  CONSTRUCTION_MENU_WIDTH = 200
38  CONSTRUCTION_MENU_ITEM_H = 40
39  CONSTRUCTION_MENU_PADDING = 10
40
41  # --- Ressources ---
42  RESOURCE_BASE_SCALE = 3
43
44  # --- Villes ---
45  CITY_MIN_TURN_EXTENTION_AVAILABLE = [3, 5, 8, 13, 21, 34, 53, 89]
46  CITY_EXTENSION_RADIUS              = 3      # distance max fondatrice ->
    nouvelle province
47  CITY_EXTENSION_FOOD_POP_MIN_RATION = 1.25 # ration min nourriture/
    habitant

```

C – Biomes, coûts de déplacement et modificateurs de combat

Le tableau suivant synthétise les propriétés de chaque biome telles que définies dans `core/systems/movement.py` et `core/systems/combat.py`.

TABLE 3 – Propriétés des biomes

Biome	Coût de déplacement (base)	Modificateur défense	Couleur (RGB)
Plaine	1,0	$\times 1,0$ (neutre)	(120, 200, 100)
Forêt	1,5	$\times 1,3$ (+30 %)	(30, 120, 30)
Montagne	2,0	$\times 1,5$ (+50 %)	(120, 120, 120)
Désert	1,2	$\times 0,9$ (−10 %)	(194, 178, 128)
Eau	∞ (infranchissable)	$\times 0,8$ (−20 %)	(50, 80, 200)

Les modificateurs de déplacement s'appliquent à *la place* du coût de base pour les unités qui possèdent un modificateur explicite :

TABLE 4 – Modificateurs de déplacement par type d'unité

Unité	Plaine	Forêt	Montagne	Désert	Eau
Soldat	1,0	1,5	2,0	1,2	∞
Archer	1,0	1,5	2,0	1,2	∞
Cavalerie	0,8	1,8	2,5	1,2	∞
Colon	1,0	1,0	1,0	1,0	1,0
Avion	1,0	1,0	1,0	1,0	1,0
Bateau	1,0	1,0	1,0	1,0	1,0

Les valeurs en gras indiquent un écart par rapport au coût de base du biome.

D – Ressources, constructions et productions

Types de ressources. Chaque tuile peut porter une ressource parmi trois niveaux (1, 2, 3) : GOLD, FOOD, WOOD, IRON, STONE (plus NONE). Le niveau détermine le multiplicateur de production de la construction associée.

TABLE 5 – Constructions : coûts et bonus de production

Construction	Coût	Ressource cible	Bonus produit
Ferme	5 bois	FOOD n	food += n (ou +1 par défaut)
Scierie	5 bois	WOOD n	wood += n (ou +1 par défaut)
Mine	2 pierre + 3 bois	IRON n / GOLD n / STONE n	iron += n , stone += 1 ou gold += n , stone += 1 ou stone += n
Route	1 pierre + 1 bois	–	movement bonus = 1 (réduction du coût de passage)

Une tuile peut accueillir au maximum **une route** et **un bâtiment** simultanément.

E – Algorithme de bruit de Perlin (utils/noise.py)

L'implémentation est extraite et simplifiée depuis la bibliothèque `noise` (non maintenue depuis 2018), en ne conservant que la partie 2D nécessaire à la génération de carte.

Principe général. Le bruit de Perlin génère une valeur continue en $[-1, 1]$ pour tout point (x, y) du plan réel. Le résultat est *pseudo-aléatoire* mais *cohérent spatialement* (les points proches ont des valeurs proches), ce qui produit des formes organiques.

Étapes de calcul (`_noise(x, y)`).

1. **Localisation de cellule.** On identifie la cellule entière $\lfloor x \rfloor, \lfloor y \rfloor$ contenant le point, puis on calcule les coordonnées relatives $x' = x - \lfloor x \rfloor \in [0, 1]$ et $y' = y - \lfloor y \rfloor \in [0, 1]$.
2. **Courbe d'atténuation (quintic).** Pour éviter les discontinuités visibles, les coordonnées relatives sont lissées par la courbe de Ken Perlin :

$$f(t) = 6t^5 - 15t^4 + 10t^3$$

3. **Vecteurs de gradient.** Les 4 coins de la cellule sont hachés via la table de permutation `PERM[512]` (table standard de Ken Perlin, doublée pour éviter les débordements d'indice). Chaque hash est converti en l'un des 8 vecteurs de gradient 2D.
4. **Interpolation bilinéaire.** Les produits scalaires $\text{grad} \cdot (x', y')$ aux 4 coins sont interpolés avec les valeurs atténuées f_x et f_y .

Bruit fractal (`perlin_noise`). Pour obtenir un terrain naturel, on superpose plusieurs couches (*octaves*) :

$$\text{total} = \sum_{k=0}^{N-1} a_k \cdot \text{noise}(f_k x, f_k y), \quad f_k = \text{lacunarity}^k, \quad a_k = \text{persistence}^k$$

Le résultat est normalisé par la somme des amplitudes $\sum a_k$. La carte utilise plusieurs octaves pour superposer des détails fins sur la structure globale des continents.

Listing 5 – Signature de `perlin_noise` (utils/noise.py)

```

1 def perlin_noise(x, y,
2     octaves=1,
3     persistence=0.5,    # amplitude diminue a chaque octave
4     lacunarity=2.0,     # frequence double a chaque octave
5     repeatx=1024, repeaty=1024,
6     base=0):
7     ...

```

F – Captures d’écran annotées

FIGURE 8 – Diversité des unités en jeu : soldat, cavalerie, avion, bateau (naviguant sur l’eau) et colon avec bâtiment



JOUEUR 1	
Tour 51	
RESSOURCES	
 FOOD	+436
 WOOD	+41
 STONE	+90
 IRON	+407
 GOLD	+167

FIGURE 9 – Barre latérale : ressources du joueur 1 au tour 51 (nourriture, bois, pierre, fer, or)